

Introduction to SQL

SQL Pros and Cons

- Declarative!
 - Say *what* you want, not *how* to get it
- Implemented widely
 - With varying levels of efficiency, completeness
- Constrained
 - Not targeted at Turing-complete tasks (not a complete programming language)
- General-purpose and feature-rich
 - many years of added features
 - extensible: callouts to other languages, data sources

SQL Language

- Two sublanguages:
 - DDL – Data Definition Language
 - Define and modify schema
 - DML – Data Manipulation Language
 - Queries can be written intuitively.
- RDBMS responsible for efficient evaluation.
 - Choose and run algorithms for declarative queries
 - Choice of algorithm must not affect query answer.

Create Table Construct

- An SQL relation is defined using the **create table** command:

```
create table  $r$  ( $A_1 D_1, A_2 D_2, \dots, A_n D_n,$   

    (integrity-constraint1),  

    ...,  

    (integrity-constraintk))
```

- r is the name of the relation
- each A_i is an attribute name in the schema of relation r
- D_i is the data type of values in the domain of attribute A_i

- Example:

```
create table branch
  (branch_name    char(15) not null,
   branch_city   char(30),
   assets         integer)
```

Example Database

Sailors

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

Boats

<u>bid</u>	bname	color
101	Nina	red
102	Pinta	blue
103	Santa Maria	red

Reserves

<u>sid</u>	bid	day
1	102	9/12/2015
2	102	9/13/2015

The SQL DDL

```
CREATE TABLE Sailors (  
    sid INTEGER,  
    sname CHAR(20),  
    rating INTEGER,  
    age FLOAT,  
    primary key (sid)  
);
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

Basic Domain Types in SQL

- **char(*n*)**. Fixed length character string, with user-specified length *n*.
- **varchar(*n*)**. Variable length character strings, with user-specified maximum length *n*.
- **int**. Integer (a finite subset of the integers that is machine-dependent).
- **smallint**. Small integer (a machine-dependent subset of the integer domain type).
- **numeric(*p,d*)**. Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric(3,1)**, allows 44.5 to be stored exactly, but not 444.5 or 0.32)
- **real, double precision**. Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(*n*)**. Floating point number, with user-specified precision of at least *n* digits.

Create Table Construct

- An SQL relation is defined using the **create table** command:

create table r ($A_1 D_1, A_2 D_2, \dots, A_n D_n$,
(integrity-constraint₁),
...,
(integrity-constraint_k))

- r is the name of the relation
- each A_i is an attribute name in the schema of relation r
- D_i is the data type of values in the domain of attribute A_i
- The allowed integrity constraints include:
 - **Primary key ($A_{j1}, A_{j2}, \dots, A_{jm}$):** states that attributes $A_{j1}, A_{j2}, \dots, A_{jm}$ constitute the primary key for relation r : non-null and unique.
 - **foreign key ($A_m, \dots, A_n$)** references r
 - **Check(P):** states that every record in the relation must satisfy the specified predicate P , e.g., $P = \text{“age} > 10\text{”}$
 - **not null**

CREATE TABLE COMMAND

- For the purposes of the following, assume the existence of these two relations:

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- create table Emp (SSN int,
 name char(15),
 age int,
 salary int,
 dno int,
 primary key (SSN),
 check (salary > 24000));
- create table Dept (dno int,
 dname char(15),
 floor int,
 mgrssn int,
 primary key dno,
);

The SQL DDL

```
CREATE TABLE Sailors (  
    sid INTEGER,  
    sname CHAR(20),  
    rating INTEGER,  
    age FLOAT  
);
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

The SQL DDL

```
CREATE TABLE Sailors (  
    sid INTEGER,  
    sname CHAR(20),  
    rating INTEGER,  
    age FLOAT,  
    PRIMARY KEY (sid));
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

```
CREATE TABLE Boats (  
    bid INTEGER,  
    bname CHAR (20),  
    color CHAR(10),  
    PRIMARY KEY (bid));
```

<u>bid</u>	bname	color
101	Nina	red
102	Pinta	blue
103	Santa Maria	red

The SQL DDL

```
CREATE TABLE Sailors (  
    sid INTEGER,  
    sname CHAR(20),  
    rating INTEGER,  
    age FLOAT,  
    PRIMARY KEY (sid));
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

```
CREATE TABLE Boats (  
    bid INTEGER,  
    bname CHAR (20),  
    color CHAR(10),  
    PRIMARY KEY (bid));
```

<u>bid</u>	bname	color
101	Nina	red
102	Pinta	blue
103	Santa Maria	red

```
CREATE TABLE Reserves (  
    sid INTEGER,  
    bid INTEGER,  
    day DATE,  
    PRIMARY KEY (sid, bid, day))
```

<u>sid</u>	<u>bid</u>	<u>day</u>
1	102	9/12
2	102	9/13

The SQL DDL

```
CREATE TABLE Sailors (  
  sid INTEGER,  
  sname CHAR(20),  
  rating INTEGER,  
  age FLOAT,  
  PRIMARY KEY (sid));
```

```
CREATE TABLE Boats (  
  bid INTEGER,  
  bname CHAR (20),  
  color CHAR(10),  
  PRIMARY KEY (bid));
```

```
CREATE TABLE Reserves (  
  sid INTEGER,  
  bid INTEGER,  
  day DATE,  
  PRIMARY KEY (sid, bid, day),  
  FOREIGN KEY (sid) REFERENCES Sailors
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

<u>bid</u>	bname	color
101	Nina	red
102	Pinta	blue
103	Santa Maria	red

<u>sid</u>	<u>bid</u>	<u>day</u>
1	102	9/12
2	102	9/13

The SQL DDL

```
CREATE TABLE Sailors (  
  sid INTEGER,  
  sname CHAR(20),  
  rating INTEGER,  
  age FLOAT,  
  PRIMARY KEY (sid));
```

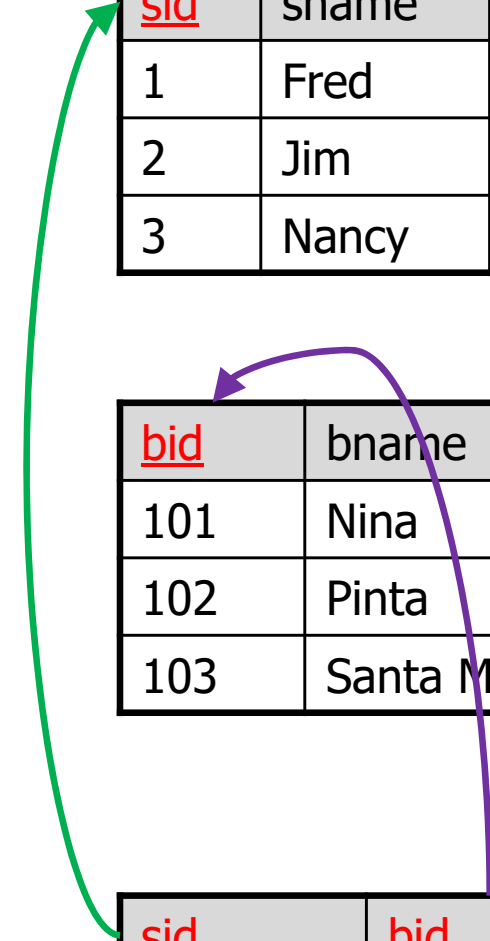
```
CREATE TABLE Boats (  
  bid INTEGER,  
  bname CHAR (20),  
  color CHAR(10),  
  PRIMARY KEY (bid));
```

```
CREATE TABLE Reserves (  
  sid INTEGER,  
  bid INTEGER,  
  day DATE,  
  PRIMARY KEY (sid, bid, day),  
  FOREIGN KEY (sid) REFERENCES Sailors,  
  FOREIGN KEY (bid) REFERENCES Boats);
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

<u>bid</u>	bname	color
101	Nina	red
102	Pinta	blue
103	Santa Maria	red

<u>sid</u>	<u>bid</u>	<u>day</u>
1	102	9/12
2	102	9/13



The SQL DDL

```
CREATE TABLE Sailors (  
  sid INTEGER,  
  sname CHAR(20),  
  rating INTEGER,  
  age FLOAT,  
  PRIMARY KEY (sid));
```

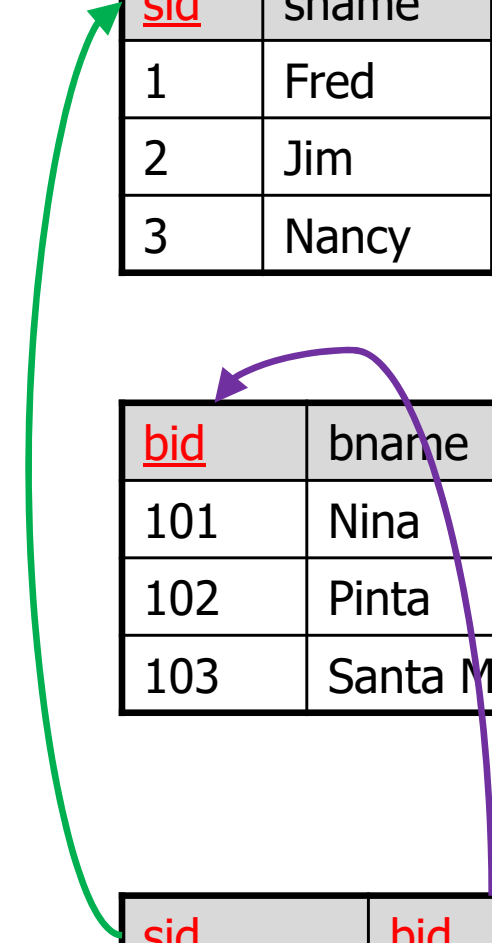
- Foreign key references a table
 - Via the primary key of that table
- Need not share the name of the referenced primary key

```
CREATE TABLE Reserves (  
  sid INTEGER,  
  bid INTEGER,  
  day DATE,  
  PRIMARY KEY (sid, bid, day),  
  FOREIGN KEY (sid) REFERENCES Sailors  
  FOREIGN KEY (bid) REFERENCES Boats);
```

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

<u>bid</u>	bname	color
101	Nina	red
102	Pinta	blue
103	Santa Maria	red

<u>sid</u>	<u>bid</u>	<u>day</u>
1	102	9/12
2	102	9/13



The SQL DML Sailors

<u>sid</u>	sname	rating	age
1	Fred	7	22
2	Jim	2	39
3	Nancy	8	27

- Find all 27-year-old sailors:

```
SELECT *  
FROM Sailors AS S  
WHERE S.age=27;
```

- To find just names and ratings,
replace the first line:

```
SELECT S.sname, S.rating
```


Updates to tables

- **Insert**
 - **insert into** *Emp* **values** (111223333,'Smith', 38, 66000);
- **Delete**
 - Remove all tuples from the *student* relation
 - **delete from** *Emp*
- **Drop Table**
 - **drop table** *Emp*
- **Alter**
 - **alter table** *r* **add** *A D*
 - where *A* is the name of the attribute to be added to relation *r* and *D* is the domain of *A*.
 - All exiting tuples in the relation are assigned *null* as the value for the new attribute.
 - **alter table** *r* **drop** *A*
 - where *A* is the name of an attribute of relation *r*
 - Dropping of attributes not supported by many databases.

Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

- A_i represents an attribute
 - R_i represents a relation
 - P is a predicate.
- The result of an SQL query is a relation.

teaches

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

[illegible]

SELECT COMMAND

- The select command has the following syntax:
select [**distinct**] target-list
from tuple variable list
where qualification
[**order by** target list subset]
[**having** set-qualification]
the items in square brackets ([]) are electives.
- Qualification list can be a boolean combination (and, or, not) of either selection and join clauses.
- A *selection clause* is a comparison between an indexed tuple variable and a constant. A comparison operator might be $\{=, \neq, \leq, \geq, >, <\}$.

ssnum	name	age	salary	dno
123	Mary	22	65000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	35000	1

dno	dname	floor	mgrssn
1	Shoe	1	323
2	Toy	1	323
3	Men	2	323
4	Women	2	324

SELECT COMMAND (Cont...)

- A *join clause* is a comparison between two indexed tuple variable. Once again, a comparison operator might be $\{=, \neq, \leq, \geq, >, <\}$. In the following example, $e.dno=d.dno$ is a join clause while $d.dname='Toy'$ is a selection clause:

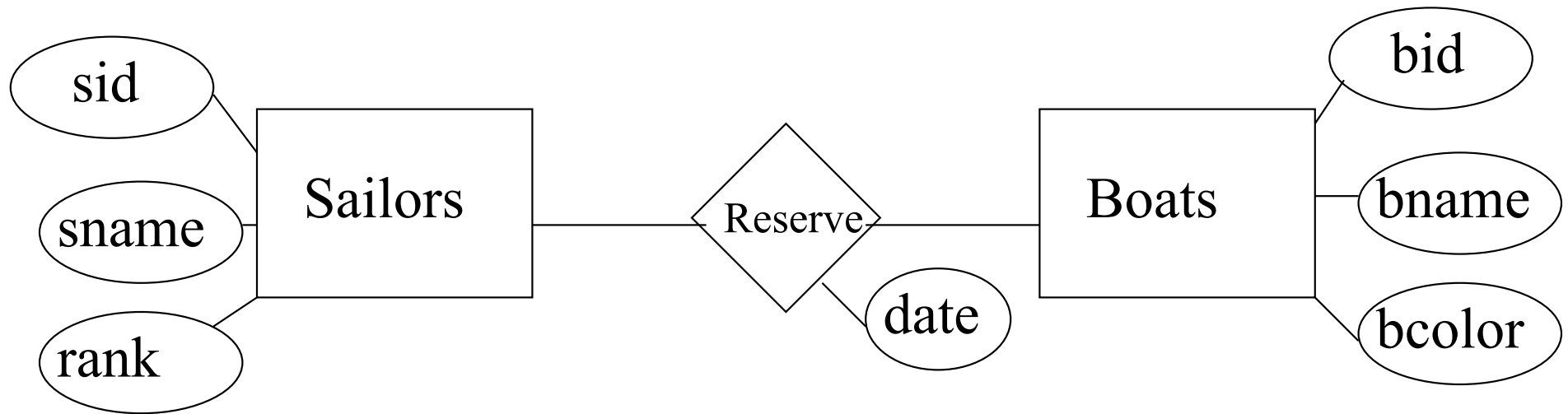
```
select e.name  
from Emp e, Dept d  
where e.dno = d.dno and d.dname = 'Toy'
```

- The results of a select query is a relation.
- To eliminate duplicates from the result, use the key word distinct in the target list. For example, the following query shows all distinct salaries above 50,000 with no duplicates:

```
select distinct e.salary  
from Emp e  
where e.salary > 50,000
```

REVIEW

- Assume a yacht club that maintains a reservation system to enable its sailors to reserve boats. For each sailor, this reservation system maintains their sailor id, name, and rank. For each boat, the system maintains a boat id, its color, and name. Sailors may reserve different boats for different dates.



Sailors (**sid**, sname, rank)

Boats(**bid**, bname, color)

Reserve (**sid**, **bid**, **date**)

REVIEW

- Query: Retrieve the sailors with a rank higher than 5.

REVIEW

- Query: Retrieve the sid of those sailors who have reserved a red boat.
- Query: Retrieve the name of those sailors who have reserved a red boat.

REVIEW

- Query: Retrieve the sid of those sailors with at least one reservation.

Query: Retrieve the name of those sailors who have reserved both a red and a green boat.

REVIEW

Query: Retrieve those sailors who have reserved either a red or a green boat.